

Desenvolvedor(a) Mobile [Flutter]

T1 - MIS7 - QP 300h Síncrono

Mini-Projeto Avaliativo - Módulo 01 - Semana 07

SUMÁRIO

1. CONTEXTUALIZAÇÃO	1
2. DESAFIO	1
3. RESULTADOS ESPERADOS (ENTREGA)	2
4. REQUISITOS DAS TAREFAS	3
4.1. ORGANIZAÇÃO DO PROJETO	3
4.2. BASE DE DADOS SIMULADA	3
4.3. REQUISITOS FUNCIONAIS	5
RF01 – Transformar mapas em objetos	5
RF02 – Tratar campos nulos	6
RF03 – Remover espaços desnecessários	6
RF04 – Converter valor monetário para número	7
RF05 – Converter horas para número inteiro	7
RF06 – Exibir todas as tarefas convertidas	8
RF07 – Filtrar tarefas por status	8
RF08 – Somar valores das tarefas concluídas	9
RF09 – Calcular média de valor das tarefas pendentes	9
RF10 – Calcular total de horas por status	9
RF11 – Identificar tarefas com dados incompletos	10
RF12 – Exibir status únicos usando Set	10
RF13 – Criar classe base e classe filha	10
RF14 – Aplicar encapsulamento	12
RF15 – Gerar relatório final	12
4.4. REQUISITOS TÉCNICOS	13
4.5. ORGANIZAÇÃO COM KANBAN E GITHUB	14
4.6. GRAVAÇÃO DE VÍDEO	16
5. CRITÉRIOS DE AVALIAÇÃO	16
6. CHECKLIST FINAL DE ENTREGA	17
7. MODELO DE README.MD PARA OS ALUNOS	18

1. CONTEXTUALIZAÇÃO

O desenvolvimento mobile exige que as pessoas programadoras saibam lidar com dados vindos de diferentes fontes. Em aplicações reais, é comum que um app receba informações de uma API, transforme esses dados em objetos da aplicação, valide campos obrigatórios, trate valores nulos, converta textos em números e gere informações úteis para a tela ou para regras de negócio.

No ecossistema Flutter, a linguagem Dart é a base para escrever a lógica da aplicação. Antes de criar interfaces completas, telas, navegação e integração com APIs reais, é fundamental

dominar a transformação de dados, a modelagem com classes, o uso de listas, mapas, funções, estruturas de decisão, estruturas de repetição e tratamento de exceções.

A documentação oficial do Dart apresenta exatamente essa base: todo programa Dart inicia por uma função `main()`, variáveis podem usar inferência de tipo, há suporte a estruturas de controle como `if`, `for` e `while`, funções com sintaxe tradicional e arrow functions, classes com atributos, construtores e métodos, além de herança.

Outro ponto essencial para mobile é a manipulação de coleções. Dart trabalha com `List`, `Set` e `Map`, e o `Map` representa dados em pares de chave e valor, estrutura muito parecida com o formato recebido de APIs e arquivos JSON. Por isso, este mini-projeto simula um cenário comum no dia a dia de uma pessoa desenvolvedora mobile: receber uma lista de dados "sujos" e transformá-la em objetos confiáveis para análise.

2. DESAFIO

Você deverá desenvolver um mini-projeto em Dart puro, chamado:

TaskReport Dart: Simulador de Tratamento de Dados de Tarefas Mobile

O objetivo do projeto é simular o recebimento de dados de tarefas vindos de uma API. Esses dados virão em uma estrutura de `List<Map<String, dynamic>>`, contendo inconsistências comuns em sistemas reais, como:

- valores monetários em texto, por exemplo: "R\$ 15,00";
- campos com espaços extras, por exemplo: "concluída ";
- campos nulos;
- textos com letras maiúsculas e minúsculas misturadas;
- valores numéricos que precisam ser convertidos;
- status diferentes para cada tarefa.

A partir dessa lista inicial, você deverá:

1. transformar os mapas em objetos Dart;
2. tratar campos nulos;
3. remover espaços desnecessários;
4. converter valores monetários para número;
5. filtrar tarefas por status;
6. somar valores de tarefas conforme o status;
7. calcular médias;
8. identificar tarefas com problemas;
9. gerar um relatório final no console;
10. aplicar Programação Orientada a Objetos.

O que este projeto deve simular:

O projeto simula uma situação real de desenvolvimento mobile:

"O aplicativo recebeu dados de tarefas de uma API. Antes de exibir ou calcular qualquer informação, o sistema precisa transformar esses dados em objetos, corrigir inconsistências e gerar um resumo confiável."

O que o projeto **não deve ter**, ou seja, está fora do escopo deste mini-projeto:

- Flutter UI;

- Widgets;
- telas;
- formulários visuais;
- navegação entre páginas;
- consumo real de API;
- HTTP;
- banco de dados;
- Firebase;
- packages externos;
- armazenamento local;
- autenticação;
- deploy;
- testes automatizados;
- extensions, pois não estão no recorte de conteúdos informado.

A entrega deve ser feita com **Dart puro**, executando o código pelo ambiente utilizado em aula, como VS Code, DartPad ou outro ambiente já orientado pelo docente.

3. RESULTADOS ESPERADOS (ENTREGA)

Ao final do mini-projeto, o estudante ou squad (grupo de até 3 alunos) deverá entregar:

1. Um repositório público no GitHub com o projeto.
2. Um arquivo principal em Dart, preferencialmente chamado main.dart.
3. Um arquivo README.md explicando a solução.
4. Um quadro Kanban com as tarefas planejadas e concluídas.
5. Um histórico mínimo de commits no GitHub.
6. Um vídeo de até 5 minutos demonstrando o funcionamento do projeto.
7. Os links submetidos na tarefa do AVA.

Entrega no AVA

O arquivo README.md, o link do repositório GitHub, o link do vídeo e o link do Kanban deverão ser submetidos na tarefa:

- **Módulo 01 - Mini-Projeto Avaliativo**
- Presente na **Semana 07** do AVA.
- Prazo de entrega: **01/06/26 até às 12h (meio-dia)**

Links obrigatórios

O estudante deverá enviar no AVA:

- link do repositório público no GitHub;
 - link do vídeo no Google Drive com permissão de leitor para qualquer pessoa com o link ou via YouTube como "não listado" ou outra maneira que seja possível visualizar o vídeo via link;
 - link do quadro Kanban utilizado no projeto.
-

4. REQUISITOS DAS TAREFAS

4.1. ORGANIZAÇÃO DO PROJETO

A estrutura mínima do projeto deverá ser:

```
taskreport-dart/  
├── main.dart  
├── README.md  
├── planejamento/  
│   └── tarefas-kanban.md
```

Não é obrigatório criar várias pastas. O foco deste mini-projeto é a aplicação correta da lógica, da manipulação de dados e da Programação Orientada a Objetos com Dart.

4.2. BASE DE DADOS SIMULADA

O projeto deverá partir de uma lista de mapas simulando o retorno de uma API.

Caso o docente disponibilize uma lista oficial em arquivo, o estudante deverá utilizar a lista enviada pelo docente. Caso contrário, poderá usar a lista abaixo como base.

Dart:

```
final List<Map<String, dynamic>> dadosTarefas = [  
  {  
    'id': 1,  
    'titulo': 'Corrigir bug login ',  
    'responsavel': 'Ana',  
    'status': 'concluida',  
    'prioridade': 'alta',  
    'valor': 'R$ 120,00',  
    'horas': '2',  
  },  
  {  
    'id': 2,  
    'titulo': 'Criar tela de perfil',  
    'responsavel': ' Bruno ',  
    'status': 'em andamento',  
    'prioridade': 'media',  
    'valor': 'R$ 250,50',  
    'horas': '5',  
  },  
  {  
    'id': 3,  
    'titulo': null,  
    'responsavel': 'Carla',  
    'status': 'pendente',  
    'prioridade': 'baixa',
```

```

    'valor': 'R$ 80,00',
    'horas': null,
  },
  {
    'id': 4,
    'titulo': ' Ajustar navegação ',
    'responsavel': null,
    'status': 'concluida',
    'prioridade': 'alta',
    'valor': 'R$ 150,75',
    'horas': '3',
  },
  {
    'id': 5,
    'titulo': 'Revisar regras de negócio',
    'responsavel': 'Daniel',
    'status': 'cancelada',
    'prioridade': 'media',
    'valor': 'R$ 0,00',
    'horas': '0',
  },
  {
    'id': 6,
    'titulo': 'Implementar validação de dados',
    'responsavel': 'Eduarda',
    'status': 'concluida',
    'prioridade': 'alta',
    'valor': 'R$ 200,00',
    'horas': '4',
  },
  {
    'id': 7,
    'titulo': 'Organizar documentação',
    'responsavel': 'Felipe',
    'status': 'pendente',
    'prioridade': 'baixa',
    'valor': 'R$ 90,00',
    'horas': '2',
  }
];

```

4.3. REQUISITOS FUNCIONAIS

RF01 – Transformar mapas em objetos

O estudante deverá transformar cada item da lista dadosTarefas em um objeto da classe Tarefa.

A classe deverá conter, no mínimo:

```

class Tarefa {
  int id;
  String titulo;
  String responsavel;
  String status;
  String prioridade;
  double valor;
  int horas;

  Tarefa({
    required this.id,
    required this.titulo,
    required this.responsavel,
    required this.status,
    required this.prioridade,
    required this.valor,
    required this.horas,
  });
}

```

O estudante deverá criar uma forma de receber um `Map<String, dynamic>` e transformar em objeto. Pode ser por meio de:

- construtor nomeado;
- função separada;
- método estático;
- ou outra solução vista em aula.

Exemplo:

```

Tarefa converterMapParaTarefa(Map<String, dynamic> item) {
  return Tarefa(
    id: item['id'],
    titulo: item['titulo'] ?? 'Sem título',
    responsavel: item['responsavel'] ?? 'Não informado',
    status: item['status'] ?? 'sem status',
    prioridade: item['prioridade'] ?? 'sem prioridade',
    valor: converterValor(item['valor']),
    horas: converterHoras(item['horas']),
  );
}

```

RF02 – Tratar campos nulos

O sistema deverá tratar campos nulos. Exemplos:

Campo	Valor nulo deve virar
-------	-----------------------

titulo	"Sem título"
responsavel	"Não informado"
status	"sem status"
prioridade	"sem prioridade"
valor	0.0
horas	0

O uso de operadores de tratamento de nulo será aceito, desde que tenha sido abordado em aula. Caso contrário, o estudante poderá usar if-else.

RF03 – Remover espaços desnecessários

O sistema deverá limpar textos com espaços extras. Exemplo:

```
" Corrigir bug login " → "Corrigir bug login"
" Bruno " → "Bruno"
```

Pode ser usado o método `trim()`.

RF04 – Converter valor monetário para número

O sistema deverá converter textos como:

```
"R$ 120,00"
"R$ 250,50"
"R$ 0,00"
```

Para números do tipo double:

```
120.00
250.50
0.00
```

Exemplo de lógica esperada:

```
dart:

double converterValor(dynamic valor) {
```

```

if (valor == null) {
    return 0.0;
}

String valorTexto = valor.toString();
valorTexto = valorTexto.replaceAll('R\\$', '');
valorTexto = valorTexto.replaceAll(' ', '');
valorTexto = valorTexto.replaceAll(',', '.');

return double.tryParse(valorTexto) ?? 0;
}

```

RF05 – Converter horas para número inteiro

O sistema deverá converter o campo horas para int. Exemplos:

```

"2" → 2
"5" → 5
null → 0

```

Exemplo:

```

dart:

int converterHoras(dynamic horas) {
    if (horas == null) {
        return 0;
    }

    return
int.tryParse(horas.toString()) ?? 0;
}

```

RF06 – Exibir todas as tarefas convertidas

O sistema deverá exibir no console todas as tarefas convertidas em objetos. Exemplo de saída esperada:

```
ID: 1
```


Título: Corrigir bug login
Responsável: Ana
Status: concluída
Prioridade: alta
Valor: R\$ 120.0
Horas: 2

Esse requisito pode usar:

- for;
 - while;
 - do-while;
 - forEach;
 - ou outro recurso visto em aula.
-

RF07 – Filtrar tarefas por status

O sistema deverá exibir tarefas conforme o status. Status mínimos:

- concluída;
- em andamento;
- pendente;
- cancelada.

Exemplo:

Tarefas concluídas:

- Corrigir bug login
- Ajustar navegação
- Implementar validação de dados

Pode ser usado where, for, if-else ou outra solução vista em aula.

RF08 – Somar valores das tarefas concluídas

O sistema deverá calcular o valor total das tarefas com status **concluído**. Exemplo:

Total de tarefas concluídas: R\$ 470.75

Pode ser usado:

- for;
- while;
- reduce;
- fold, se tiver sido trabalhado;
- ou outra solução equivalente.

Como o tópico informado menciona métodos avançados como `map`, `every`, `find`, `filter` e `reduce`, o estudante pode adaptar esses conceitos usando os métodos equivalentes do Dart, como `map`, `where`, `firstWhere`, `every` e `reduce`.

RF09 – Calcular média de valor das tarefas pendentes

O sistema deverá calcular a média de valor das tarefas com status pendente. Exemplo:

Média de valor das tarefas pendentes: R\$ 85.00

Regra:

$\text{média} = \text{soma dos valores das tarefas pendentes} / \text{quantidade de tarefas pendentes}$

Caso não exista nenhuma tarefa pendente, o sistema deverá exibir:

Não existem tarefas pendentes para calcular média.

RF10 – Calcular total de horas por status

O sistema deverá calcular o total de horas por status. Exemplo:

Horas por status:
concluída: 9 horas
em andamento: 5 horas
pendente: 2 horas
cancelada: 0 horas

Esse requisito pode usar `Map<String, int>` para armazenar o total por status.

RF11 – Identificar tarefas com dados incompletos

O sistema deverá exibir tarefas que vieram com algum dado ausente ou inválido na lista original. Exemplos de inconsistência:

- título nulo;
- responsável nulo;
- horas nulas;
- valor inválido;
- status vazio.

Exemplo de saída:

Tarefas com dados incompletos:
- ID 3: título ausente ou horas ausentes

- ID 4: responsável ausente

RF12 – Exibir status únicos usando Set

O sistema deverá exibir os status únicos encontrados na lista. Exemplo:

Status encontrados:

- concluida
- em andamento
- pendente
- cancelada

Esse requisito deverá usar **Set**. A documentação oficial do Dart apresenta Set como uma coleção de itens únicos, o que torna esse tipo adequado para listar valores sem repetição.

RF13 – Criar classe base e classe filha

O projeto deverá conter uma classe base e uma classe filha. Sugestão:

```
dart:

class ItemTrabalho {
  int id;
  String titulo;

  ItemTrabalho({
    required this.id,
    required this.titulo,
  });

  void exibirResumo() {
    print('Item $id - $titulo');
  }
}
```

Classe filha:

```
dart:

class Tarefa extends ItemTrabalho {
  String responsavel;
  String status;
```

```
String prioridade;
double valor;
int horas;

Tarefa({
  required int id,
  required String titulo,
  required this.responsavel,
  required this.status,
  required this.prioridade,
  required this.valor,
  required this.horas,
}) : super(id: id, titulo: titulo);

@override
void exibirResumo() {
  print('Tarefa $id - $titulo | Status: $status | Valor: R$ $valor');
}
}
```

Esse requisito avalia:

- classe;
- objeto;
- atributos;
- métodos;
- construtor;
- herança;
- polimorfismo;
- sobrescrita com **@override**.

Dart possui suporte a classes, construtores, métodos e herança simples com **extends**, conforme a documentação oficial da linguagem.

RF14 – Aplicar encapsulamento

O estudante deverá aplicar encapsulamento em pelo menos um atributo. Exemplo:

```
dart:

class RelatorioTarefas {
  final List<Tarefa> _tarefas;

  RelatorioTarefas(List<Tarefa> tarefas): _tarefas = tarefas;

  int get quantidadeTotal => _tarefas.length;
}
```

O objetivo é demonstrar que alguns dados podem ficar protegidos dentro da classe, sendo acessados por métodos ou getters.

RF15 – Gerar relatório final

Ao final da execução, o sistema deverá exibir um relatório consolidado. Exemplo:

RELATÓRIO FINAL DE TAREFAS
Total de tarefas analisadas: 7
Tarefas concluídas: 3
Tarefas pendentes: 2
Tarefas em andamento: 1
Tarefas canceladas: 1
Valor total das concluídas: R\$ 470.75
Média de valor das pendentes: R\$ 85.00
Total de horas concluídas: 9
Status encontrados:
concluída
em andamento
pendente
cancelada
Tarefas com dados incompletos:
ID 3 - Sem título
ID 4 - Ajustar navegação

4.4. REQUISITOS TÉCNICOS

Conteúdo	Obrigatório?	Como demonstrar
Lógica de programação	Sim	Conversões, cálculos e classificações
Dart	Sim	Código em <code>.dart</code>
Sintaxe básica	Sim	<code>main</code> , variáveis, funções e prints
Tipos de dados	Sim	<code>String</code> , <code>int</code> , <code>double</code> , <code>bool</code> , <code>List</code> , <code>Map</code> , <code>Set</code>
If-else	Sim	Validações e tratamento de status
Switch-case	Recomendado	Pode ser usado para classificar status

Conteúdo	Obrigatório?	Como demonstrar
Operador ternário	Recomendado	Pode ser usado em mensagens simples
Operadores lógicos	Sim	Validações compostas
Operadores matemáticos	Sim	Soma, média e percentuais
For, while ou do-while	Sim	Iteração sobre tarefas
Funções	Sim	Conversão de valor, conversão de horas, filtros e relatórios
Arrow Functions	Sim	Uso em <code>map</code> , <code>where</code> , <code>every</code> ou função simples
Lists	Sim	Lista de tarefas
Maps	Sim	Dados simulados da API
Sets	Sim	Status únicos
Map de chave e valor	Sim	<code>Map<String, dynamic></code>
Métodos avançados	Sim	Usar pelo menos 3 entre <code>map</code> , <code>where</code> , <code>firstWhere</code> , <code>every</code> , <code>reduce</code>
Classes	Sim	Classe base e classe filha
Objetos	Sim	Tarefas convertidas
Atributos	Sim	Campos da tarefa
Métodos	Sim	Exibir resumo e gerar relatório
Construtores	Sim	Inicialização dos objetos
Modificadores de acesso	Sim	Usar <code>_atributo</code> em pelo menos uma classe
Encapsulamento	Sim	Atributo privado com getter ou método
Herança	Sim	<code>Tarefa extends ItemTrabalho</code>
Polimorfismo	Sim	Sobrescrever método com <code>@override</code>
GitHub	Sim	Repositório público
GitHub Desktop	Não	Pode ser usado para commits e push
GitFlow simplificado	Sim	Branches simples

Conteúdo	Obrigatório?	Como demonstrar
Kanban	Sim	Quadro com tarefas do projeto

4.5. ORGANIZAÇÃO COM KANBAN E GITHUB

O estudante ou squad (grupo de até 3 pessoas) deverá organizar o trabalho em um quadro Kanban. As colunas mínimas obrigatórias são:

Backlog	A Fazer	Em Andamento	Concluído
---------	---------	--------------	-----------

Ferramentas aceitas:

- Trello;
- GitHub Projects;
- Notion;
- planilha;
- arquivo no README;
- imagem do quadro.

Tarefas sugeridas para o Kanban:

- Criar repositório no GitHub
- Criar arquivo main.dart
- Criar README.md
- Adicionar lista de mapas simulando API
- Criar classe ItemTrabalho
- Criar classe Tarefa
- Criar conversão de Map para objeto
- Tratar campos nulos
- Remover espaços extras dos textos
- Converter valor monetário para double
- Converter horas para int
- Exibir todas as tarefas convertidas
- Filtrar tarefas por status
- Somar valores das tarefas concluídas
- Calcular média das tarefas pendentes
- Calcular horas por status
- Listar status únicos com Set
- Identificar tarefas com dados incompletos
- Aplicar encapsulamento
- Aplicar herança
- Aplicar @override
- Aplicar tratamento de exceções
- Gerar relatório final
- Revisar código

- Atualizar README
- Gravar vídeo
- Enviar links no AVA

Branches mínimas

Para projeto individual : main develop feat/conversao-dados feat/relatorio-tarefas docs/readme-video	Para projeto em squad (até 3 pessoas): main develop feat/modelagem-poo feat/tratamento-dados feat/calculos-relatorios docs/readme-video
--	--

Commits mínimos

Projeto individual : mínimo de 5 commits	Projeto em squad (até 3 pessoas): mínimo de 8 commits
---	--

Exemplos de commits

feat: cria estrutura inicial do projeto feat: adiciona lista de mapas simulando api feat: cria classes ItemTrabalho e Tarefa feat: implementa conversao de valor e horas feat: adiciona filtros por status feat: gera relatorio final de tarefas docs: atualiza readme com instrucoes fix: corrige tratamento de campos nulos
--

4.6. GRAVAÇÃO DE VÍDEO

Além do desenvolvimento das tarefas você deverá gravar um vídeo, com tempo **máximo de 5 minutos**, abordando os seguintes questionamentos:

- Qual é o objetivo do sistema?
- Como o sistema simula o retorno de uma API?
- Como os mapas foram transformados em objetos?
- Como executar o projeto?
- Como você organizou as tarefas antes de começar a desenvolver?
- Qual ferramenta de Kanban foi utilizada?
- Quais branches foram criadas e qual o objetivo de cada uma?
- Onde foram usados List, Map e Set?

- Onde foram usados classes, objetos, construtores, encapsulamento, herança e @override?
- O que faltou no código que poderia ser melhorado?

Regras do vídeo:

- Tempo máximo: 5 minutos.
- Pode ser gravado na vertical ou na horizontal.
- Deve aparecer o rosto do estudante.
- O webcam, tela e áudio devem estar compreensíveis.
- O projeto deve ser demonstrado funcionando.
- O vídeo deve estar no Google Drive com permissão de leitor para qualquer pessoa com o link. Ou via YouTube como Não Listado. Ou qualquer maneira possível de assistir o vídeo via link;
- O link do vídeo deve estar no README e também ser enviado no AVA.

5. CRITÉRIOS DE AVALIAÇÃO

A tabela abaixo apresenta os critérios avaliados durante a correção do projeto.

A nota varia de **0 a 10 pontos**. Este mini-projeto possui peso de 25% sobre a avaliação do módulo.

Serão desconsiderados e **receberão nota 0 (zero)** os projetos que apresentarem **plágio** de soluções encontradas na internet ou de outros colegas.

O estudante deve consultar materiais, documentação, exemplos e ferramentas de apoio, desde que compreenda, adapte e consiga explicar o código entregue. Lembre-se: Você está livre para utilizar outras soluções como base, mas não é permitida a cópia.

Apresentação do Projeto – 4,00 pontos

Nº	Critério de Avaliação	0	Parcial	Máximo
1	Entregou o repositório completo no GitHub com um arquivo README completo: com objetivo, como executar o código, com estrutura das pastas, tecnologias utilizadas, links necessários (vídeo, etc).	0	0,75	1,50
2	Entregou o vídeo apresentando o projeto, demonstrando o funcionamento do código e sistema, explicando as decisões técnicas do código, mostrando o kanban utilizado e o fluxo seguido para concluir o projeto.	0	0,75	1,50
3	Organizou o repositório do projeto no GitHub, usando commits e branches assertivas com nomenclaturas adequadas ao GitFlow.	0	0,50	1,00

Subtotal: **4,00 pontos** 🖐️

Desenvolvimento do Projeto – 6,00 pontos

Nº	Critério de Avaliação	0	Parcial	Máximo
4	Transformou corretamente <code>List<Map<String, dynamic>></code> em objetos Dart	0	0,50	1,00
5	Tratou os campos nulos, espaços extras, conversão de valores e horas	0	0,75	1,50
6	Filtrou por status, soma valores, calcula média e totaliza horas	0	0,75	1,50
7	Usou corretamente Lists, Maps, Sets, funções, loops e métodos de coleção	0	0,50	1,50
8	Aplicou POO com classes, objetos, métodos, construtores, encapsulamento, herança e <code>@override</code>	0	0,50	1,50

Subtotal: **6,00 pontos** 🖐️

Total: 10,00 pontos

6. CHECKLIST FINAL DE ENTREGA

Antes de enviar no AVA, confira:

- ☐ [] Criei o repositório público no GitHub
- ☐ [] Criei o arquivo main.dart
- ☐ [] Criei o README.md
- ☐ [] Criei o quadro Kanban
- ☐ [] Usei uma `List<Map<String, dynamic>>` como base de dados simulada
- ☐ [] Transformei os Maps em objetos Dart
- ☐ [] Criei uma classe base
- ☐ [] Criei uma classe filha
- ☐ [] Usei construtor
- ☐ [] Usei atributos e métodos
- ☐ [] Usei herança
- ☐ [] Usei `@override`
- ☐ [] Apliquei encapsulamento
- ☐ [] Tratei campos nulos
- ☐ [] Removi espaços extras dos textos
- ☐ [] Converti valor monetário para double
- ☐ [] Converti horas para int
- ☐ [] Exibi todas as tarefas convertidas
- ☐ [] Filtrei tarefas por status
- ☐ [] Somei valores das tarefas concluídas

- ☐ [] Calculei média das tarefas pendentes
 - ☐ [] Calculei total de horas por status
 - ☐ [] Liste status únicos com Set
 - ☐ [] Identifiquei tarefas com dados incompletos
 - ☐ [] Gerei relatório final no console
 - ☐ [] Fiz commits no GitHub
 - ☐ [] Usei branches
 - ☐ [] Gravei vídeo de até 5 minutos
 - ☐ [] Coloquei o vídeo no Google Drive com permissão correta
 - ☐ [] Enviei os links no AVA
-

7. MODELO DE README.MD PARA OS ALUNOS

Markdown:

```
# TaskReport Dart
```

```
## Sobre o projeto
```

```
O TaskReport Dart é um simulador de tratamento de dados de tarefas em Dart.
```

```
O projeto simula o retorno de uma API por meio de uma lista de mapas, transforma esses dados em objetos e gera relatórios com base nos status das tarefas.
```

```
## Objetivo
```

```
Praticar os principais conceitos do Módulo 01:
```

- lógica de programação;
- linguagem Dart;
- tipos de dados;
- estruturas de decisão;
- estruturas de repetição;
- funções;
- arrow functions;
- Lists;
- Maps;
- Sets;
- métodos de coleção;
- classes;
- objetos;
- atributos;
- métodos;

- construtores;
- encapsulamento;
- herança;
- polimorfismo com @override;
- GitHub;
- GitFlow;
- Kanban.

Como executar

Execute o arquivo `main.dart` no ambiente utilizado em aula.

Exemplo:

```
```bash
dart run main.dart
```

Ou execute pelo DartPad, caso orientado pelo docente.